

Lectura 2 - Database Caching Strategies Using Redis

Estudiante: Priscilla Romero Barquero

Carné: 2023332718

1. Explique los desafíos que se enfrentan en la construcción de sistemas distribuidos que requieren baja latencia (de acuerdo con la lectura)

Slow processing queries

Como una base de datos en disco debe leer toda la información desde almacenamiento físico, la velocidad a la hora de que el disco quiere recuperar la información por ejemplo cuando quiere hacer alguna consulta el motor de la base de datos tiene que buscar toda la información, más los tiempos de procesamiento que se sumen antes de devolverlos hace que se aumenten los tiempos de respuesta.

Escalabilidad costosa

Escalar para una gran cantidad de lecturas puede ser costoso independientemente de si es una base de datos NoSQL basada en disco o en una base de datos relacional. Además, algunas veces para escalar un sistema se puede requerir varias réplicas de lectura de la base de datos para igualar la capacidad de un solo nodo de caché en memoria en términos de solicitudes por segundo y esto solo implica costos adicionales.

Acceso a los datos

Las bases de datos basadas en almacenamiento en disco son buenas para modelar relaciones pero no son buenas para acceder a los datos y esto hace que acceder a los datos se necesite una estructura o vista específica para simplificar este proceso y aumentar rendimiento.

2. Explique los tipos de caching, ¿Cuál considera es el más adecuado?

Database-integrated caches

Ofrecen una caché integrada que se administra dentro del motor de base de datos y cuenta con capacidades de escritura directa integradas, es decir la base de datos incluye una caché en memoria interna. La base de datos actualiza su caché automáticamente cuando cambian los datos subyacentes.

El problema es su tamaño y sus capacidades, ya que suelen estar limitadas a la memoria disponible que le asigna la instancia de la base de datos.

Local Caches

Este tipo de caching almacena los datos de uso frecuente dentro de la memoria de la aplicación para agilizar la recuperación de datos que son consultados siempre y eliminar el tráfico asociado a esa recuperación.

Una desventaja es que al estar dentro de una aplicación no se puede compartir información con otras cachés por ser nodos individuales y esto afecta en entornos distribuidos porque los nodos pueden estar desactualizados con respecto a otros. Además, al trabajar localmente, es muy difícil trabajar con aplicaciones

de varios servidores y si hay interrupciones, los datos de la caché local se pierden y deben rehidratarse, lo que anula la caché.

Remote caches/Side cache

Este tipo se basa en instancias independientes dedicadas a almacenar los datos en caché en memoria, almacenadas en servidores específicos para el caso y suelen basarse en key-value NoSQL.

Una ventaja es que la latencia promedio de una solicitud a una caché remota es de menos de un milisegundo, mucho más rápido que una solicitud a una base de datos en disco. Además, son ideales para entornos distribuidos porque se conectan a todos los servidores utilizados.

¿Cuál puede ser la mas adecuada?

La mejor opción puede Side Cache debido a las ventajas que tiene como el hecho de que la latencia es mucho más rápido que una memoria en disco además por el hecho de que tiene la oportunidad de compartir la información con otros nodos permitiendo coherencia entre ellos.

3. Explique las diferencias entre los patrones de caching.

Cache-aside/lazy loading

Este tipo de patrón se basa en actualizar la caché después de solicitar la data. Por ejemplo, si la aplicación tiene que leer información de la base de datos, primero se revisa la caché para saber si la data está disponible. Si la información está disponible entonces se retorna y se emite al solicitante pero si la data no está disponible entonces se consulta a la base de datos, la caché se actualiza con nueva información y se emite al solicitante.

La ventaja de este tipo es que la caché tiene solo la información que se solicita haciendo su tamaño algo rentable.

Write-Through

Este tipo de patrón se basa en actualizar inmediatamente la caché después que la base de datos se actualice, entonces si el proceso de aplicación, lote o backend actualiza la base de datos inmediatamente lo hace en caché también.

La ventaja es que los datos siempre están en caché esto permite mejor rendimiento en la aplicación y mejor experiencia de usuario, pero al almacenarse ahí siempre hace que sea costosa.

4. Explique que es cache eviction, ¿Por qué es relevante para el rendimiento de un sistema?

Cache Eviction se refiere a cuando la memoria caché se llena, está sobrecargada o supera el límite de memoria máxima permitido y se necesita liberar espacio para guardar nuevos datos.

Se mencionan diferentes políticas a seguir para desalojar los datos y es vital elegir la adecuada para no cometer errores de eliminar datos importantes por ejemplo. El cache eviction es importante porque al estar lleno puede que las consultas a la caché duren más tiempo y más bien lo que se busca es lo contrario.

Además, si se presentan muchos evictions significa que debe escalar verticalmente u horizontalmente su clúster (conseguir nodos con mayor consumo de memoria) para guardar datos adicionales.